

Athens University of Business and Economics
Business Administration School
Department of Management Science and Technology
Software Engineering in Practice



6^o Semester

Markos Balaskas (8230101)

Professor: Diomidis Spinellis

Athens, 2026

Introduction

This report documents the engineering process behind the design, containerization, and cloud-native deployment of a Node.js REST API (my-api) as part of the Software Engineering in Practice assignment. The goal of the assignment was to build a complete production-inspired pipeline, from writing a Dockerfile to orchestrating the application on a local Kubernetes cluster using Minikube.

The pipeline was built incrementally across four tasks:

- Task 1 — Containerizing the application using Docker
- Task 2 — Automating the build and publish process via a GitHub Actions CI/CD pipeline
- Task 3 — Deploying the application to Minikube using Kubernetes manifests including a ConfigMap, Secret, Deployment, and Service
- Task 4 — Validating the deployment through port-forwarding and documenting the full process

The repository can be found under the GitHub link:
https://github.com/markoulinoss/seip_assignment_1_2026 Markos Balaskas

CI/CD Proof:

All workflows		Filter workflow runs				
Showing runs from all workflows						
5 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Add readMe CI/CD Pipeline #5: Commit 19b9f99 pushed by markoulinoss	main			8 minutes ago 26s	...
✓	Fix typo in deployment CI/CD Pipeline #4: Commit a397e15 pushed by markoulinoss	main			May 22, 8:20 PM GMT+3 20s	...
✓	Test: change repository read and write settings and removed manual ... CI/CD Pipeline #3: Commit 17ffb7e pushed by markoulinoss	main			May 22, 6:34 PM GMT+3 21s	...
✓	Fix typo in deployment and add permissions for token to be pushed ... CI/CD Pipeline #2: Commit c6a2744 pushed by markoulinoss	main			May 22, 6:23 PM GMT+3 32s	...
✗	Add tasks 1-3, containerization, CI/CD Actions workflow, and k8s CI/CD Pipeline #1: Commit 31e7cbf pushed by markoulinoss	main			May 22, 6:18 PM GMT+3 16s	...



markoulinos

markoulinoss · he/him

Student at Athens University of Business
and Economics

Type: All ▾ 🔍 Search packages...

Visibility: All ▾

Sort by: Most downloads ▾

📦 1 package

📦 my-api

Published last week by [markoulinos](#) in [markoulinoss/seip_assignment_1_2026_Markos_Balaskas](#)

📄 0

Cluster State Proof:

```
C:\Users\chris>kubectl get all -n default
NAME                                READY    STATUS    RESTARTS   AGE
pod/my-deployment-7f65c85d55-btf2m 1/1      Running   0           14m
pod/my-deployment-7f65c85d55-jknw8 1/1      Running   0           14m
pod/my-deployment-7f65c85d55-lwsv6 1/1      Running   0           14m

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
service/kubernetes                  ClusterIP      10.96.0.1     <none>       443/TCP    61m
service/my-service                  ClusterIP      10.101.210.48 <none>       80/TCP     14m

NAME                                READY    UP-TO-DATE  AVAILABLE   AGE
deployment.apps/my-deployment       3/3      3            3           14m

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/my-deployment-7f65c85d55 3          3          3        14m

C:\Users\chris>kubectl get configmap,secret
NAME                                DATA    AGE
configmap/kube-root-ca.crt         1        62m
configmap/my-config                 2        15m

NAME                                TYPE     DATA    AGE
secret/my-secret                    opaque   1        15m
```

Commands run:

- `kubectl get all -n default`
- `kubectl get configmap,secret`

Application Verification Proof:

```
C:\Users\chris>curl http://localhost:8080
{"message":"Welcome to the Software Engineering in Practice Assignment Cluster!","environment":"production"}
C:\Users\chris>curl http://localhost:8080/health
{"status":"Healthy"}
C:\Users\chris>curl http://localhost:8080/secure-config
{"status":"Authorized","injected_secret_suffix":"*****MENT"}
C:\Users\chris>
```

Commands run:

- `http://localhost:8080/`
- `http://localhost:8080/healthy`
- `http://localhost:8080/secure-config`

AI Usage & Future Engineering Report

AI Integration

For the purposes of this assignment, Claude was utilized as an AI assistant. It was primarily utilized for debugging processes in GitHub Actions, for understanding Kubernetes concepts such as ConfigMaps, Secrets, Service and Deployment and Minikube startup failures on Windows. It was also very helpful in providing trusted sources for the documentation of the use of Kubernetes which helped me understand the complex concepts such as

- <https://kubernetes.io/docs/concepts/configuration/secret/>
- <https://kubernetes.io/docs/concepts/security/secrets-good-practices/>
- <https://kubernetes.io/docs/concepts/configuration/configmap/>

It was also used for generating boilerplate code (such as initial YAML manifests) and for further in-depth explanation of what each line of code consists of. The code was later reviewed line-by-line and modified to match the project's structure and my naming conventions. Beyond syntax, Claude explained the practical meaning behind the code and helped me make practical decisions, hence Opaque, Kubernetes default, being used for secrets whereas in Kubernetes documentation many other types like basic auth or tls.

Utility Analysis

AI assistance was very helpful in many aspects. Not so much in syntax debugging, seeing how built-in tools of VS Code can catch and correct such errors but more so in debugging and error diagnosis.

1. **Real time error diagnosis:** The most valuable use was diagnosing real time errors in GitHub Actions workflow. When the CI/CD pipeline failed with denied: installation

not allowed to Create organization package, it was able to point out the root cause the initial missing “packages: write permission”. After some deeper investigation, I changed the repository settings to “Read and write permissions” open to all, so that workflows have read and write permissions in the repository for all scopes. Even though this is not a recommended practice (https://docs.github.com/en/actions/tutorials/authenticate-with-github_token#modifying-the-permissions-for-the-github_token), for the purposes of this project, I think it will suffice since it is not a real-world enterprise system.

2. **Concept-to-Code Translation:** Kubernetes YAML manifests are verbose and interconnected. Claude was particularly useful in explaining how concepts connect in practice — for example, how the selector.matchLabels in a Deployment must exactly match the labels in the pod template, or how envFrom pulls all keys from a ConfigMap directly into the container's environment.
3. **ImagePullBackOff Pod Failures:** After successfully starting Minikube and applying Kubernetes manifests with “kubectl apply -f k8s/”, all 3 pods were stuck in ImagePullBackOff status:

NAME	READY	STATUS	RESTARTS	AGE
my-deployment-764fdb854-bpmp4	0/1	ImagePullBackOff	0	81s
my-deployment-764fdb854-bvtxj	0/1	ImagePullBackOff	0	81s
my-deployment-764fdb854-qlsp4	0/1	ImagePullBackOff	0	81s

This meant that Kubernetes could not pull the Docker image from GHCR. The root cause was that the CI/CD pipeline had not successfully pushed the image to GHCR yet, so there was nothing for Kubernetes to pull. The CI/CD pipeline was failing at the push step with:

- denied: installation not allowed to Create organization package

Claude identified the fix needed:

Adding a permissions block to the workflow granting packages: write to the built-in GITHUB_TOKEN

I choose to, for this specific project, enable Read and write permissions under repository Settings → Actions → General → Workflow permissions.

Once the pipeline passed successfully and the image was available in GHCR, the pods automatically transitioned from ImagePullBackOff to Running after reapplying the manifests:

4. **Windows-Specific Minikube Issues:** Another issue that AI was helpful with was Minikube failing to start on Windows 11. The error I encountered was:

- open C:\Users\chris\.minikube\machines\minikube\id_rsa: Access is denied

Minikube's SSH key file (id_rsa) had become locked by the Windows file system, preventing Minikube from reading its own configuration. Running “minikube delete –purge” also failed because Windows wouldn't allow deletion of the locked file:

- unlinkat C:\Users\chris\.minikube\machines\minikube\id_rsa: Access is denied

I honestly had no idea what to do afterwards to resolve this problem, because my only idea had been to delete and re-install and my knowledge of this kind of troubleshooting is limited. AI suggested I run Powershell as an administrator and to take ownership of the file and grant full permissions before deleting the file

- takeown /f "C:\Users\chris\.minikube\machines\minikube\id_rsa"
- icacls "C:\Users\chris\.minikube\machines\minikube\id_rsa" /grant chris:F
- Remove-Item -Recurse -Force "C:\Users\chris\.minikube"
- minikube start

This was a case where generic Linux-based documentation would not have helped.

Friction Points

Thankfully, I didn't encounter very serious problems or dead ends neither with local network routing issues nor Minikube VM context errors. The most prominent problem was the GHCR push error, but that was fairly easily overcome. Also, the fact that minikube delete –purge did not work initially because of the locked file permissions that Claude had not anticipated, so I had to do some further investigation into the error output. Overall, it was a very constructive process.

Future Architectural Outlook

If I had an extra week or was tasked with scaling this pipeline for a real-world enterprise system in the future, I would follow most if not all the recommendations like replacing port-forwarding with an Ingress Controller or establishing a true GitOps workflow with ArgoCD.

More specifically, the current kubectl port-forward approach is mostly a development tool, and it dies when the terminal closes. In reality, this would be replaced with an Ingress Controller such as a NGINX Ingress that could also enable multiple services under one domain and load balancing across replicas. Currently, accessing the my-api service requires manually running 'kubectl port-forward service/my-service 8080:80' every time, which dies when the terminal closes. In production, this would be replaced with the NGINX so that my-api is permanently accessible at a real domain like api.myapp.com/health and api.myapp.com/ without any manual intervention.

GitOps Workflow

It would also be beneficial to establish a GitOps Workflow with ArgoCD. The current CI/CD pipeline pushes an image to GHCR but does not automatically update the Kubernetes cluster. In a real enterprise system, ArgoCD would be introduced to implement a true GitOps workflow, where the cluster continuously watches the repository and automatically syncs any changes to the Kubernetes manifests. As a result, ArgoCD would watch the repository and automatically sync the cluster whenever the manifests change, meaning a git push to main would trigger a full end-to-end deployment of my-api with zero manual steps

Add Prometheus & Grafana Monitoring

Currently there is no visibility into how the 3 replicas of my-api are performing. Prometheus would scrape metrics from each pod, such as how many requests hit the / and /health endpoints, how much CPU and memory each replica consumes, and how many times the livenessProbe has failed and triggered a restart. Grafana would display these in a real-time dashboard, making it immediately obvious if one of the 3 replicas is underperforming.

Add Image Scanning to the CI Pipeline

Currently the ci-cd.yaml pipeline builds the my-api image from node:18-alpine and pushes it to GHCR without any security checks. A tool like Trivy would be added as a step between Build and Tag Docker Image and Push Docker Image to GHCR to scan for known vulnerabilities in the Node.js dependencies and base image. If a critical vulnerability were found, the pipeline would fail and the image would never reach the cluster.

Secrets Manager

I would also replace Kubernetes Secrets with a Proper Secrets Manager. As highlighted in the Kubernetes documentation, base64 encoding is not encryption. It works

deterministically, meaning that it always gives the same output, and it is fairly easy for someone to reverse engineer the passkey.

For example, the `API_SECRET_KEY` used in this assignment was encoded as:

```
U0VJUF9BU1NJR05NRU5U
```

Anyone who gains access to the `secret.yaml` file can instantly decode it in any terminal with a single command:

```
bashecho " U0VJUF9BU1NJR05NRU5U " | base64 --decode
```

```
# output: SEIP_ASSIGNMENT
```

One does not need special tools or any effort. This is why base64 is for encoding and not for encryption; it purely does a formatting transformation, not a security measure. If `secret.yaml` were accidentally committed to a public GitHub repository, any person who found it could decode every credential instantly. This is why I would opt for a proper secrets manager like HashiCorp Vault that stores credentials encrypted at rest, requires authentication to access, and never exposes the raw value in a file that could be committed to version control.

Also, after fully grasping the concept of Minikube with the help of AI, it is safe to say that it is obviously unsuitable for a real-world enterprise system to scale. We would probably have to migrate to a Kubernetes service from a cloud provider such as Google or Azure. Seeing that Minikube runs locally the clusters on a single machine using the laptop's CPU and RAM with their respective limitations. That's why Minikube is great for learning but could never handle real production traffic, making it unsuitable and worth mentioning the obvious but vital change.